research · 2026-08-21

a regex can pass its tests and still fail in use

Adrian Tame Jacobo, writing for Plicara Labs · 2026-08-21 ·

Editorial clarification, September 2026: this article distinguishes observational comparisons from causal explanations, and unresolved differences from equivalence. The recorded results are unchanged. The linked v1.0 whitepaper is an archived release; this article contains the updated interpretation.

> Eleven current language models wrote 450 regular expressions each, three times over, scored three ways: whether the pattern passes its tests, whether it means what the task asked for, and whether it is vulnerable to regular-expression denial of service. Then we took the safety screen, which needs no model at all, and pointed it at five more populations of regular expressions, half a million of them pulled out of shipped packages.
>
> Three findings guide the study. **Vulnerability-screening rates differ across the sampled corpora**: the forum, reuse-library, and answer-key populations screen at 13% to 20%, shipped-package patterns at 8.9%, and model outputs at 9.8%. Execution and maintenance are possible explanations, not measured causes. **7.4% of the patterns that pass their tests are exploitable**, compared with much larger penalties reported for backend code. **That 7.4% becomes 16.5% on a second benchmark**, so the measured effect depends on the corpus as well as the domain.

Claude Opus 5 produced the following pattern for a domain-name validation task. It passes every test the benchmark supplies.

```
^([a-zA-Z0-9]([a-zA-Z0-9\-]{0,61}[a-zA-Z0-9])?\.)+(com|org|net|mil|edu)$
```

Under review it reads as careful work. It anchors both ends, caps label length at 63 characters, the DNS limit, and checks the top-level domain against a list.

The defect is the outer group. `(...)+` wraps a group that already contains `{0,61}`, which is the construction that makes a backtracking engine catastrophically slow. Given a long, almost-valid hostname that fails at the final character, the matcher tries exponentially many ways to divide the

input before concluding there is no match. Deployed on a signup form, the pattern is a denial-of-service vector.

This is ReDoS, well enough documented to have its own systematisation-of-knowledge paper. Nobody ships it on purpose, it shipped here because it passed its tests and because it looks careful.

We wanted to look at cases like this.

---

previous work

The benchmark already exists. The corpus we use is Re(gEx|DoS)Eval, and it comes from a 2024 paper by Mohammed Latif Siddiq, Jiahao Zhang, Lindsay Roney and Joanna C. S. Santos at Notre Dame (ICSE-NIER 2024). They collected 762 regex problems from real user posts, wrote tests and a reference answer for each, and defined the four measurements we report: does it pass its tests, is it vulnerable to ReDoS, does it denote the same language as the reference, and is it character-for-character identical. They then scored T5, Phi-1.5 and GPT-3.5-Turbo on all of it and reported which model wrote regexes that were correct *and* secure. What we are doing is running their apparatus on eleven models that did not exist when they built it, taking their composite metric apart, and pointing their safety check back at the answer key we score the models on.

Scoring correctness and security together is a live area. Four benchmarks published in the last two years do it for general code: CWEval (119 tasks over 31 CWEs, five languages), BaxBench (392 backend tasks with expert-written exploits), SecureAgentBench (105 repository-level tasks aimed at coding agents), and DualGauge (154 tasks, 10 models). They agree on the shape of the result, BaxBench finds roughly half of functionally correct backends exploitable. SecureAgentBench reports 15.2% correct-and-secure for its best agent. DualGauge sees secure-pass@1 under 12% while functional pass@1 clears 50%. The gap between "works" and "safe to ship" is large and well established.

Before any of those, Pearce et al. generated 1,689 programs in security-relevant scenarios and found about 40% vulnerable, and Perry et al. ran a user study where people with an AI assistant wrote less secure code while reporting more confidence in it.

Basically, people already knew regexes are a problem. Davis et al. measured ReDoS across the npm and PyPI ecosystems. Siddiq's own companion study looked specifically at ReDoS in LLM-generated patterns.

What the rest of this article is about:

1. Nobody had run the safety screen on the benchmark's **own human reference answers**, or against the regular expressions people ship.
2. Nobody had **audited the reference set** these metrics compare against.
3. Nobody had run any of it on the **current model population**, at least as of late 2026.

We also ran the whole measurement again on a **second benchmark**, to see which of our numbers were about regular expressions and which were about Re(gEx|DoS)Eval.

---

what we did

Each task gives a model a plain-English description, such as *"Matches 5 numeric digits, such as a zip code"*, and the model writes a pattern. It gets no worked examples and no second attempt, and every model gets the same instruction.

We ran 450 of the 762 tasks, spread evenly across the corpus so the sample is not weighted toward the easy end. Eleven current frontier and open-weights models, the cheapest costing a hundredth of the dearest, three attempts each.

Then we asked three questions about every answer:

1. **Does it work?** The pattern is run against the strings that should match and the strings that shouldn't.

2. **Does it mean the right thing?** The pattern is compared to the human answer as a *language* rather than as text, because `[0-9]+` and `[0-9][0-9]*` describe exactly the same set of strings and a benchmark comparing text would call one of them wrong.

3. **Is it safe?** The pattern is screened for the shapes that backtrack catastrophically, then attacked with strings built to trigger them.

The scoring is done by regexbench, a tool we wrote before this project and pinned to one commit for the run. It is our own instrument. The equivalence check inside it is dk.brics.automaton, which is an excellent piece of software in our estimation and deserves lots of love.

The second and third questions are what this corpus adds over the older regex benchmarks, KB13 and NL-RX, which score a candidate against a reference by language equivalence and stop there.

---

the numbers

Between **38.0% and 46.5% of answers pass their tests**, depending on the model. Between **17.1% and 23.8% survive all three questions**, and that second column is a conjunction of the three, which we take apart much further down because it does not mean what it looks like it means.

model	passes tests	survives all three	vulnerable
`kimi-k3`	46.5%	23.8%	12.9%
`qwen3.6-max-preview`	42.4%	21.6%	9.1%
`gpt-5.6-sol`	42.1%	20.9%	10.0%
`claude-opus-5`	46.1%	20.8%	13.2%
`deepseek-v4-flash-0731`	38.0%	19.8%	12.0%
`qwen3.6-plus`	39.8%	19.8%	9.8%
`glm-5.2`	42.4%	18.7%	14.2%
`gpt-5.6-terra`	42.2%	18.7%	12.0%
`gpt-5.6-luna`	39.2%	18.5%	11.6%
`claude-sonnet-5`	40.7%	18.0%	10.9%
`gemini-3.1-flash-lite`	38.7%	17.1%	12.0%

The one number in that table to hold on to is the last column. We found **390 generations that passed every test and were exploitable**, out of 5,269 that passed: 7.4%, or 144 of the 2,051 model-task pairs where anything passed at all. The interesting thing is that they are basically ordinary: email validators, hostname validators, a pattern for matching comma-separated names.

Everything below is either an attempt to find out what that 7.4% is really a fact about, or an attempt to find out whether the column next to it can be trusted.

the human answers are more dangerous than the machine ones

The obvious conclusion at this stage is that language models write dangerous regular expressions. So we turned the same safety screen on the **human-written answers** in the benchmark, the reference patterns the corpus uses as its gold standard, written by people, for a benchmark about regular expressions. **13.6% of them are vulnerable.**

The models range from 7.3% to 10.7%. Pooled across all eleven, 9.0%.

	vulnerable
Human reference answers	**13.6%**
Best model (`qwen3.6-max-preview`)	7.3%
Worst model (`kimi-k3`)	10.7%
All models pooled	9.0%

Every model screens safer than the answer key, though how many of them do so *provably* is a smaller number. Eleven independent confidence intervals are the wrong test here: every model answered the same 450 tasks, so the comparison is paired and an unpaired interval throws away the pairing. The right test is McNemar's, run on the tasks where a model and the answer key disagree. Correcting for having run eleven of them, **six of the eleven separate at 95%**. The ones that do not separate are the most vulnerable models, which is what you would expect and is not what a row of point estimates told us.

As far as we can tell this comparison had not been run before, and it is only possible because this corpus ships human answers. The general-code benchmarks execute tests and exploits instead of comparing against a gold artifact, so they have nothing to point the screen at.

The tempting conclusion is that dangerous regular expressions are endemic to how people write them and the models learned it from us. But it rests on one corpus, and that corpus is an answer key, not working code, so the next section goes and gets five more.

shipped packages and published examples differ

The really nice thing about this is that none of it needs a single API call. The safety screen reads patterns, so running it on somebody else's corpus costs nothing but CPU cycles. We screened the gold answers of KB13, the machine-generated patterns of NL-RX, and three corpora from Davis et al.'s artifact: half a million regular expressions **extracted from shipped packages** across npm, PyPI, Maven, CPAN, crates.io, godoc, packagist and RubyGems, half a million more **posted to Stack Overflow**, and the 3,838 patterns **published to regexlib.com** for other people to reuse.

The last two matter because Re(gEx|DoS)Eval was built from real user posts. If forum snippets are dangerous, then the benchmark's answers are dangerous for a reason that has nothing to do with benchmarks.

Raw rates are dominated by task mix. This corpus is full of validators, email and ISBN and hostname, which is exactly the shape that backtracks, while most regexes in the wild are short fragments with no opportunity to. So the table below restricts every population to anchored `^...$` patterns, which is the closest we can get to comparing similar objects:

anchored patterns only	source context	n	vulnerable
RegexLib, published for reuse	reuse library	1,684	20.1%
Stack Overflow answers	forum	4,000	17.3%
Re(gEx|DoS)Eval gold answers	benchmark	538	13.4%
our eleven models	—	3,613	**9.8%**
production code	**shipped packages**	4,000	**8.9%**

The sampled shipped-package patterns have a lower screening rate than the sampled reference and forum populations. That comparison does not establish a general ordering of human and model safety.

> **Corpus membership is observed; execution history is not.**

The forum, reuse-library, and answer-key populations screen between 13% and 20%. Patterns extracted from shipped packages screen at 8.9%, compared with 9.8% for model outputs. The

reported test does not resolve that latter difference (p = 0.20); it does not establish equivalence.

The model row is restricted the same way as every other row: we keep only the models' own anchored outputs, because the rule keys on the pattern and not on who wrote it. Putting the models in at their unrestricted rate would compare a restricted human population against an unrestricted machine one, which is exactly the confound the restriction exists to remove. Restricting them moves the models slightly *toward* the answer key and away from production code, and the conclusion holds anyway.

Execution and maintenance could help explain the lower rate in shipped packages: deployed patterns may be tested, monitored, or repaired. But this is an **observational comparison**. We did not measure whether each pattern had actually run, and forum snippets may also have been executed. Restricting to anchored patterns does not control for task complexity, ecosystem, selection, or maintenance history. Forum posts screen at 17.3%; the lower rate in the gold set could reflect selection, but it does not demonstrate how the corpus authors filtered it. A matched or longitudinal study would be needed to distinguish these explanations.

The practical implication is to screen patterns at the point of use, whether they came from a model, a forum, or a shipped package. These measurements do not justify assuming any source is safe.

---

we measured our own blind spot

Everything above rests on one screen deciding which patterns are dangerous. If that screen is blinder in shipped code than it is in showcase validators, the ordering we just published is an artifact of the tool rather than a fact about the world. Calling the screen "a lower bound" is a fair caveat for one population, but across six it is an excuse.

So we measured its blind spot, separately in each population. We took a second, independent detector, the one Davis and colleagues used for their own ecosystem study, written by different people in a different language on a different theory, and where it flagged a pattern, we took the attack string it produced, fed it to Python at growing sizes, and timed the match. A pattern counts as genuinely dangerous when the matcher measurably fails to keep up.

population	our screen caught	recall
Stack Overflow	25 of 27	92.6%

population	our screen caught	recall
regexlib.com	33 of 36	91.7%
Re(gEx|DoS)Eval gold	19 of 22	86.4%
production code	16 of 20	80.0%
NL-RX-Synth	16 of 21	76.2%
our models	11 of 15	73.3%
KB13	2 of 3	66.7%

Our screen is *worse* at production code, 80%, than at the showcase populations we compare production code against, 92%. That is precisely the direction in which a blind instrument would manufacture our result.

What rescues the finding is the size. Correct each population by its own recall and the gap between the most vulnerable read-only population and shipped code goes from 11.2 points to 10.8. The instrument's bias is real and it eats four tenths of an eleven-point gap. A differential that would have to close eleven points closes less than one.

Our own models have the second-worst recall in that table. Correcting everything by its own recall moves the models from 9.8% to 13.4% and production code from 8.9% to 11.1%, so the point-estimate gap roughly doubles. The interval on that 73.3% recall runs from 48% to 89%, making the correction uncertain. Neither the raw comparison nor this correction establishes equivalence between the populations.

a place we disagree with prior work

Siddiq's companion ReDoS study reports that LLM-generated patterns skew toward *polynomial* rather than exponential blow-up. Our models go the other way, 5.3% exponential against 3.8% polynomial pooled. The cross-corpus run says where that skew actually lives: in the anchored production sample, polynomial beats exponential 253 to 105, while the benchmark's gold answers are near even at 34 to 38.

So the polynomial skew is real, and it is a property of the regular expressions people ship, not of the ones models write. On this axis, too, model output differs from human practice instead of

reproducing it. The subcategory counts on our side are small and we make no strong claim. We would love for somebody to replicate it. Every count above is in `results/cross_corpus_redos.json`, written by a script in the repository that needs no API key to re-run.

how big is the correctness-to-security penalty, really

Back to that composite for a moment, because the number we want is inside it. It has three conjuncts, two of them run a real regex engine against real strings, while the third compares against a human's answer. Ask which conjunct is producing the gap and the split is not close:

conjunct removed	cost to the score
Safety	2.9 points
Semantic equivalence	18.9 points

87% of our headline gap comes from the equivalence term. And the equivalence term, as the audit further down shows, is 85% noise from bad reference answers and prompts that never specified the property in dispute.

Dropping it and scoring only the two criteria that never consult a human answer key, which is also the construction the general-code benchmarks use, leaves the finding that **7.4% of the regular expressions that work are ReDoS-vulnerable.**

That number is much smaller than the composite suggested, and it does not match what the rest of the field reports.

	functional	correct-and-secure
BaxBench (backends)	—	~ half of correct solutions exploitable
SecureAgentBench (repo-level agents)	higher	15.2% best agent, 9.2% mean
DualGauge (specification-only)	>50%	<12%

	functional	correct-and-secure
This work (regexes)	**38–47%**	**security removes 7.4%**

The security criterion demolishes most of what passes in those settings. In ours it takes off a sliver. We have explanations for that and no way to separate them with this data. A regex is one expression with one failure mode while a backend has many independently exploitable parts. ReDoS is a structural anti-pattern, plausibly better represented in training data than CWE-classified defects are. Our functional pass rate is also low enough that the correct subset may skew toward simple tasks with less room for catastrophic backtracking.

The correctness-to-security penalty does not transfer between domains, it has to be measured in the one you are claiming it for. That is a weaker claim than "correct code is often insecure" and a more useful one. It is also the reason a joint metric should never be published without its decomposition: ours reported a penalty of about the size the literature would predict, and almost none of it was the penalty.

yes, we ran even more benchmarks

The screen has now been on six populations, but every number that involved a model has come from one corpus. The 7.4% is a fact about Re(gEx|DoS)Eval until somebody checks it somewhere else, so we checked.

StructuredRegex is a second benchmark for the same task, and it has the one thing the older ones lack: example strings for every problem, both matching and non-matching. That is sufficient for *does it work*. Its answer key, written in a notation of its own, is never read, so this run carries no equivalence score at all. Only the two questions that held up.

	re(gex|dos)eval	structuredregex
passes its tests	30.0–41.4%	51.7–66.1%
vulnerable	7.3–10.7%	13.6–17.9%
vulnerable, of the ones that work	**7.4%**	**16.5%**

The finding holds: patterns that pass their tests and remain exploitable are not a quirk of one benchmark.

The number does not hold, it more than doubles. And the easy explanation, that the second benchmark is harder so the answers are worse, is the wrong way round: StructuredRegex is *easier*, by about twenty points. Its problems are built out of repetition and optional parts, and models answering those write more of the shape that backtracks.

The section above argued that this penalty depends on the domain, on the strength of BaxBench and SecureAgentBench doing something rather different in other languages. This is the same task, the same language, the same screen, the same eleven models, and the answer is 2.2 times apart. It is a much stronger version of the same argument, and it applies to our own number as much as to anybody else's.

The ordering moved too. Sonnet 5 comes first here and sits in the middle of the other table. Kimi K3 comes first there and eighth here. We already said a numbered list of eleven models was not defensible. This is what that looks like under test.

Anthropic's content filter refused 182 of the 622 descriptions for Opus, all of them harmless ("a string that starts with one or more digits and optionally ends with 'NU' or 'DG'"). Retrying recovered 98. The temptation is to score Opus on what survived and move on. Instead we checked what the other ten models did on the problems Opus lost, and those problems turned out to be **8.1 points harder** than average. So the leftovers flattered Opus, while counting the refusals as wrong answers punished it, both at once. Every number above is on the 513 problems all eleven models actually answered.

Why the filter fires on these at all, we can only guess. We think maybe the descriptions sit close enough to other attack surfaces, SQL injection and the like, that a filter keyed on the request blocks them; whether that is a good trade is very much not the point of this paper, but it is strange to watch it happen in live data.

---

the difficulty objection

There is an obvious deflationary reading of our 7.4%: maybe the patterns models get *right* are the easy ones, and easy patterns have less room to go wrong. If so the number is a selection effect, not a finding.

We can test that inside one corpus, which is better than comparing two corpora written by different processes. Sort the tasks by how many of the eleven models solved them, the

benchmark's own measure of how easy a task turned out to be, and look at vulnerability inside each band. On tasks six or more models solved, 7.1% of correct patterns are dangerous. On tasks only one to five solved, 10.7%.

The objection survives, weakly and against us: the number moves in the direction the objection predicts. Resampled, the difference is −3.6 points with an interval running from −12.2 to +3.8, so this corpus cannot actually establish it. What the exercise does establish is a ceiling. Even on the hardest tasks any model solved, vulnerability among the answers they got right is 10.7%, not the roughly 50% that BaxBench reports for backend code. The difficulty effect is real enough to mention and far too small to be the explanation.

---

cost differences and unresolved score differences

The eleven models span a 98× range in price.

model	survives all three	cost per request
`deepseek-v4-flash-0731`	19.8%	$0.000026
`claude-opus-5`	20.8%	$0.002514

DeepSeek's model costs **98× less** and scores a point *lower*. The reported comparison does not resolve that score difference. **The whole field fits inside seven percentage points**, but this is a statement about these measured scores, not proof of equal capability.

For this task, the cost difference is clear while the score comparison remains uncertain. Practical equivalence would require a prespecified acceptable difference and an analysis designed to test it. These results also cannot establish what the models encountered in training.

---

does it mean anything, though?

The second question, *does it mean the right thing?*, compares the model's pattern against a person's. That measures the model only if the person was right.

This is not a new worry in the abstract. Northcutt, Athalye and Mueller audited ten of the most-used test sets in machine learning and found a mean 3.3% label-error rate, enough in several cases to flip which model the benchmark said was better. Nobody had asked the question of this corpus. We have one advantage they did not: **our labels are regular expressions, so a disputed label can be settled by producing a string the reference and the description disagree about, rather than by taking a second opinion.**

We drew a random sample of sixty cases where a model passed every test but was scored as meaning something different, and worked through fourteen of them carefully. We expected to find models making subtle mistakes.

who was actually wrong	share
The human answer	36%
Neither, the prompt never said	43%
The model	21%

The task said "it just accepts only positive numbers." The human answer was `^\d+([.,]?\d+)?$`, which accepts `0`. The model wrote a pattern that excludes zero. Zero is not a positive number. The model was marked down for being right, and ground truth continues to be hard.

The task asked for a simple ISBN check, "a 10 digit number." The human answer was `^\d{9}[\d|X]$`. Inside the brackets: digit, **pipe**, X. Someone wrote `|` meaning "or", inside a character class, where it is just the pipe character. That answer accepts `000000000|` as a valid ISBN. The model wrote `^\d{9}[\dX]$`, which is what the task described, and lost.

The task asked for a pattern to "make sure commas are in the rite place." The human answer made the commas optional, so it accepts `0,000000`, defeating the only thing it was for. The model enforced comma placement and was scored as different.

In a further 43% of cases neither answer was wrong, because the question did not have one right answer. *"Matches any single upper- or lower-case letter"*: does that mean the whole string is one letter, or that a letter appears somewhere? The sentence does not say. The benchmark assumes the first. A model that assumes the second is not making a mistake, but it is counted as one.

Separately, a third of all the disagreements came down to `\d` versus `[0-9]`, which differ only on characters like `٣`, the Arabic-Indic three. That is a technical difference with no practical consequence in almost any real use.

Put together: **the model is clearly at fault in far less of this than the metric counts against it.**

That reading comes from fourteen cases judged by us, on a benchmark we were using to make a point. The direction is clear enough to act on and every judgement is written down in the repository so it can be argued with.

---

which of the three metrics held up

The two questions that never look at the human answer key, *does it work* and *is it safe*, held up. They run a real regex engine against real strings, and no defect in an answer key can corrupt them.

The question that compares against a human did not, and it failed twice over. In our sample it was 85% noise from bad answer keys and ambiguous prompts. And 48% of the credit it *awarded* went to patterns nobody checked, because the composite scores "cannot tell" as a pass and the equivalence engine could not parse one side or the other. The tidy story about that would be backreferences, where equivalence really is undecidable and a model could earn credit for putting its answer beyond checking. The real answer is the parser: of 471 such credits, backreferences account for twenty.

> **The metrics that consult a human answer key behaved differently from the metrics that do not, and only the second kind survived scrutiny.**

Reading the disagreements is what exposes the first failure and decomposing the metric is what exposes the second, and neither is expensive. Fourteen cases and one afternoon of arithmetic were enough.

The two surviving metrics are not unconditionally clean either, and we know how dirty they are because we measured it. The safety screen misses between 7% and 33% of what is genuinely dangerous, and misses unevenly across the populations we compare. That is a real limitation, quantified, and it is small enough not to move the ordering. Knowing the size of a limitation is a different thing from having caveated it.

---

what we are not claiming

We are not publishing a leaderboard.

Comparing the models properly, which means accounting for the fact that they all answered the same questions, nine of the fifty-five possible pairwise comparisons come out distinguishable. Unpaired intervals on the same data resolve exactly one, which is a ninefold difference from choosing the right test. The test is not ours either: the paired bootstrap is Berg-Kirkpatrick, Burkett and Klein, following Koehn's bootstrap-resampling protocol for machine translation, and using it instead of unpaired intervals is standard advice.

Even corrected, the best model separates from six of the other ten and the middle of the table does not separate at all. There is a structural reason. **62% of the tasks give every single model the identical result**, either right across the board or wrong across the board. Only 162 of the 421 tasks every model answered in full do any work telling these models apart.

Bands are defensible. A numbered list from one to eleven is not, and anyone who re-ran this and got a different order would be right to. So here are the bands:

- **Ahead of at least one model, behind none.** `claude-opus-5`, `kimi-k3`, `qwen3.6-max-preview`.
- **No comparison resolves either way.** `deepseek-v4-flash-0731`, `gpt-5.6-sol`.
- **Behind at least one model, ahead of none.** `claude-sonnet-5`, `gemini-3.1-flash-lite`, `glm-5.2`, `gpt-5.6-luna`, `gpt-5.6-terra`, `qwen3.6-plus`.

Read those as statements about each model, not as three rungs. The bands are not ordered against each other either: `claude-opus-5` is in the first and `glm-5.2` in the last, and those two are not distinguishable. Only `kimi-k3` separates from more than two others. Every pairwise interval is in `results/sweep/paired_intervals.json`.

We are not claiming any of these percentages travel. We have run two corpora and the vulnerability rate among working patterns came out 2.2× apart, so treat every figure in this article as a fact about the benchmark it was measured on until somebody runs a third. *Does it mean the right thing* is still single-corpus and always will be, because no other benchmark for this task ships an answer key in ordinary regex syntax.

We also could not test contamination. This corpus was published in 2024 and built from public forum posts, so it is plausibly inside every one of these models' training data. Sainz et al. argue that contamination has to be measured per benchmark rather than waved away, and we agree, and we cannot do it. We have no private task set. This is the largest hole in the work.

---

how we got here, in order

Three of the results above replaced an earlier reading of the same data, and the checks that replaced them were all cheap. Putting them in one place, since they are the part of this that transfers to work that has nothing to do with regular expressions.

The composite came before the decomposition. We built the three-way conjunction because that is what the joint-benchmark literature reports, and it gave a gap of about the size that literature would predict. 87% of it was the equivalence term, and the equivalence term was 85% noise. A composite is no more reliable than its worst conjunct and it does not tell you which conjunct is binding, so publish the decomposition or do not publish the composite.

The human baseline came before the cross-corpus comparison. One corpus suggested that model outputs screened safer than human reference answers. Extending the analysis exposed differences between kinds of artifacts. Execution and maintenance are hypotheses for those differences, not identified causes; the extension required no further model calls.

Reference-independent did not mean corpus-independent. The 7.4% consults no answer key, which makes it more trustworthy than the composite and does not make it general. StructuredRegex said 16.5%. Treat an effect size as local to its corpus until a second one disagrees, and the second corpus is usually cheaper than the first.

Two estimator bugs were sitting in per-sample records, invisible in every aggregate. The standard `pass@k` shortcut returns 1.0 when fewer than `k` samples came back, which is correct on the domain the estimator is defined on and silently wrong below it, so tasks that returned one sample were scoring as full passes. Fixing it moved Opus from second to fourth. Separately, the timing oracle was labelling every timeout exponential, which is precisely the error we criticise in the section above. Both were found by reading raw output rather than summaries.

---

reproduction

Every response from every model is committed to the repository. The scores compute from those files and nothing else, so reproduction costs nothing and needs no API key:

```
git clone https://github.com/plicara/regexeval-2026
cd regexeval-2026
make setup
make score RUN=sweep
```

We verified this by wiping to a clean checkout, reinstalling everything, re-downloading the corpus and re-scoring from scratch. Every number came out identical. A version of that check runs automatically on every change, so what is published cannot drift away from the evidence behind it.

That guarantee used to stop at the repository's edge. Reviewers of the first draft found nine arithmetic inconsistencies, and a second round found five more; almost every one was a sentence quoting a table, or a table maintained by hand, drifting when the underlying number moved. Being right about the data and wrong in the write-up is still being wrong. So the write-up is generated too: every table in the paper is emitted from committed results, every figure the prose quotes comes from a generated macro, and a check refuses the build if a percentage appears in the text that no script produced. Running that check on ourselves immediately found two stale numbers neither reviewer had caught.

Every request was pinned to one named provider and refused substitution, because the router that sits in front of these models will otherwise serve the same model from different companies at different numerical precision, at which point the measurement is of the router. A survey of AI-safety codebases using one such router found 31 of 32 did not pin the provider. All eleven of our models were served by exactly the endpoint they were pinned to.

On every run, three fake answers ride through the scoring alongside the real ones: a known-good pattern that must pass, a known-bad one that must fail, and a known-dangerous one that must be flagged. If any of them misbehaves the run is discarded. A scorer quietly returning zeros is indistinguishable from a set of models that all failed, unless a known answer is planted and checked on the way through.

---

what is next

The large open question is whether letting these models think helps. We have a twelve-task comparison, which is too small to mean anything, and one firm number: turning reasoning on made each request **15.7x more expensive**. On the easiest task in the corpus, matching a single digit, one model spent 1,571 tokens of hidden reasoning before answering `^[0-9]$`. With reasoning off it gave the same answer in ten tokens.

Whether that expense buys accuracy is worth measuring properly. It is probably its own article.

---

references

Natural-language-to-regex generation, where language equivalence against a reference became the standard criterion:

- Kushman & Barzilay. *Using Semantic Unification to Generate Regular Expressions from Natural Language.* NAACL-HLT 2013. aclanthology
- Locascio, Narasimhan, DeLeon, Kushman & Barzilay. *Neural Generation of Regular Expressions from Natural Language with Minimal Domain Knowledge.* EMNLP 2016. aclanthology

The benchmark and metrics we used:

- Siddiq, Zhang, Roney & Santos. *Re(gEx|DoS)Eval: Evaluating Generated Regular Expressions and their Proneness to DoS Attacks.* ICSE-NIER 2024. doi:10.1145/3639476.3639757 · corpus
- Siddiq, Zhang & Santos. *Understanding Regular Expression Denial of Service (ReDoS): Insights from LLM-Generated Regexes and Developer Forums.* ICPC 2024. doi:10.1145/3643916.3644424
- Chen et al. *Evaluating Large Language Models Trained on Code.* 2021. arXiv:2107.03374. Source of the pass@k estimator.
- Ye, Chen, Dillig & Durrett. *Benchmarking Multimodal Regex Synthesis with Complex Structures.* ACL 2020. aclanthology. The second corpus.

Joint correctness-and-security benchmarking:

- Peng et al. *CWEval.* LLM4Code 2025. arXiv:2501.08200
- Vero et al. *BaxBench.* ICML 2025. arXiv:2502.11844
- Chen et al. *SecureAgentBench.* 2025. arXiv:2509.22097
- Patir et al. *DualGauge.* 2025. arXiv:2511.20709
- Pearce et al. *Asleep at the Keyboard?* IEEE S&P 2022. arXiv:2108.09293
- Perry et al. *Do Users Write More Insecure Code with AI Assistants?* CCS 2023. doi:10.1145/3576915.3623157

ReDoS:

- Davis et al. *The Impact of ReDoS in Practice.* ESEC/FSE 2018. doi:10.1145/3236024.3236027
- Davis, Michael IV, Coghlan, Servant & Lee. *Why Aren't Regular Expressions a Lingua Franca?* ESEC/FSE 2019. artifact, the source of the 537,806 production regexes screened here.

- Bhuiyan, Çakar, Burmane, Davis & Staicu. *SoK: A Literature and Engineering Review of ReDoS.* AsiaCCS 2025. arXiv:2406.11618

Measurement:

- Northcutt, Athalye & Mueller. *Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks.* NeurIPS D&B 2021. arXiv:2103.14749
- Berg-Kirkpatrick, Burkett & Klein. *An Empirical Investigation of Statistical Significance in NLP.* EMNLP-CoNLL 2012. aclanthology
- Koehn. *Statistical Significance Tests for Machine Translation Evaluation.* EMNLP 2004. aclanthology
- Dror et al. *The Hitchhiker's Guide to Testing Statistical Significance in NLP.* ACL 2018. aclanthology
- Sainz et al. *NLP Evaluation in Trouble.* Findings of EMNLP 2023. aclanthology

Tooling, disclosed because two of these are ours:

- regexbench, the scorer. Prior work by this lab, pinned to a single commit for this run.
- dk.brics.automaton, Anders Møller's finite-state automata library, which does the language-equivalence check.
- Not Pinning Your OpenRouter Provider Might Invalidate Your Research, the survey behind the serving-provider discipline.

The full technical write-up, with the statistics, the failure taxonomy and every adjudicated case, is in `paper/main.tex` in the repository.

The whitepaper: Passing Is Not Shipping (PDF). Data, method, and the re-run command: regexeval-2026, tagged `v1.0` as published. Scoring by regexbench 0.4.1.